Homework 5: Greedy

Guideline

Homework chatbot. Open the homework Gem here. Use this Gem when working through the AI-related sections. The bot is already curated with the right answers and hints to provide the right level of guidance to help you learn.

Showing your honest attempt can only help. There is no advantage in cheating. I give 100% credit on attempts, even if they are incorrect. Cheating on homework will also only hurt you. If you cheat, I won't be able to give you the proper feedback to help you learn, nor adjust the difficulty of the quizzes. You will also likely fail on the in-person mastery quizzes, which test concepts tackled in the homework.

Collaboration: Only after solo attempt. You may collaborate and use external resources, but (1) you are encouraged to do the homework **alone first** and (2) you must **attribute** these external resources when you do use them.

AI: Follow guideline. The problems will be very clear on when you are allowed to use AI. Please follow them. Here is why: (1) Unregulated usage of AI harms learning (2) Even in the age of AI, your AI-independent ability matters. The **only** AI tool you should use on this homework is the homework-specific Gemini Gem that I have created for you.

Submission. Upload all files to the Homework 5 assignment on Canvas. **5 files total:**

1. **One answer document** containing your answers to every question below. Name it `lastname-hw5.pdf` (or `.txt`). LaTeX isn't required -- plain text, Google Docs → Download as PDF, or Word → Save as PDF are all fine.
2. **Four AI chat transcript PDFs**, one for each AI-check step. Start a **new chat** in the Gem for each step, so that each transcript covers only that step:
    - `arrows-b.pdf`, `arrows-c.pdf`
    - `cookies-b.pdf`, `cookies-c.pdf`

Accepted file formats: `.pdf` for everything; `.txt` is also fine for the answer document. Do **not** paste chat content into your answer document, and do **not** submit a link to a Google Doc -- upload the file itself.

Due: Sunday, October 25, 2026 at 11:59pm. See the syllabus for the late-day policy.

Points: 22 total. Remember, I give full points to every question that was attempted, and 0 only for a question left blank. What matters is that you have **thought** through the problems enough to prepare yourself for the quizzes.

1. Non-overlapping intervals (2 pts total)

Given an array of intervals `(start, end)`, return the **minimum number of intervals you have to remove** so that the ones left over do not overlap each other.

Leetcode link

Intervals that merely touch at a point do **not** count as overlapping -- `[1,2]` and `[2,3]` can both stay.

Here are some example input and output:

#	`intervals`	answer	why
1	`[[1,2],[2,3],[3,4],[1,3]]`	1	remove `[1,3]` and the rest are disjoint
2	`[[1,2],[1,2],[1,2]]`	2	two of the three copies have to go
3	`[[1,2],[2,3]]`	0	they only touch, so nothing needs removing

No AI on this problem at all.

(a) Map it to something we already solved (1 pt)

In 3 to 5 sentences, describe how this problem relates to a problem we solved in lecture. Name the lecture problem, and give the exact relationship between its answer and this one. Your answer should make it obvious why solving one immediately solves the other.

(b) Code (1 pt)

Code it up on Leetcode. Turn in your final code, pasted into your answer document, and paste your Leetcode submission URL showing a successful submission.

2. Minimum number of arrows to burst balloons (9 pts total)

Balloons are laid out on a number line. Balloon `i` spans `[start_i, end_i]`. You shoot arrows straight up from a coordinate `x`; an arrow at `x` bursts **every** balloon whose span contains `x`, and it keeps travelling forever, so one arrow can burst any number of balloons.

Return the **minimum number of arrows** needed to burst all the balloons.

Leetcode link

Here are some example input and output:

#	`points`	answer	why
1	`[[10,16],[2,8],[1,6],[7,12]]`	2	shoot at `x = 6` and `x = 11`
2	`[[1,2],[3,4],[5,6],[7,8]]`	4	nothing overlaps, so every balloon needs its own
3	`[[1,2],[2,3],[3,4],[4,5]]`	2	shoot at `x = 2` and `x = 4`

(a) Step 1. Brainstorm greedy strategies (1 pt)

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own.

Write down **at least two** different greedy strategies you could imagine using here. For each one, say in a single line what rule decides where the next arrow goes.

You are not trying to be right yet. Step 1 is a brainstorm -- the point is to have candidates to knock down.

(b) Step 2. Prove, or kill it

(i) Kill this one with a counterexample (1 pt)

A classmate proposes this strategy:

> Repeatedly fire an arrow at the coordinate that bursts the **most balloons that are still unburst**. Break ties arbitrarily. Stop once every balloon is burst.

This one is tempting -- every shot does as much damage as it possibly can. It is still wrong.

Find a **counterexample**: a small set of balloons where this fires more arrows than necessary. Give the balloons, say how many arrows the rule uses, and how many are actually needed.

Because the rule breaks ties arbitrarily, your example has to beat it **no matter which tie it picks**.

You can do it with **six** balloons.

Hint. Aim for a case where **two** arrows are enough, but the coordinate that bursts the most balloons is not one of those two.

(ii) Prove the one that works (1 pt)

The strategy that does work is:

> Sort the balloons by **end** coordinate. Walk through them in that order. Whenever you meet a balloon that the current arrow does not burst, fire a new arrow at **that balloon's right edge**.

Prove it with an **exchange argument**. Fill in this skeleton:

1. Let `g_1, ..., g_k` be the coordinates greedy shoots at, in order, and let `o_1, ..., o_m` be the shots of some optimal solution, also in ascending order. Assume for contradiction that these two sequences disagree.
2. Walk along both until they first disagree, at position `i`. So `g_j = o_j` for every `j < i`, and `g_i != o_i`.
3. Because of how greedy chooses, we know that `g_i` ... compared to `o_i`. **Q:** What is the inequality here? >=? <=? Why?
4. Now replace `o_i` with `g_i`, and so we are considering OPT', which is `o_1, ..., o_{i-1}, g_i, o_{i+1}, ... o_m`. Argue that this sequence is valid and is as good as OPT. Consider the next interval that's not burst yet after we shoot at `o_1, ..., o_{i-1}`, and this interval is the next interval that's not burst yet, and has the smallest end time. Now, `g_i` shoots at this interval's end time, whereas `o_i` shoots further to the left. **Q:** Could it be that `o_i` destroy more balloons than `g_i` at this moment?
5. Conclude.

Fill in steps 3 to 5.

(iii) AI plan check

Use the homework chatbot to check your counterexample from (i) and your proof from (ii). Here are some example prompts:

- **Verifying.** "Here is the problem (copy paste). Here is my counterexample and my exchange argument (copy paste). Are these right?"
- **Brainstorming.** "I can't find a counterexample for the most-balloons-per-shot rule. Can you give me a hint?"
- **Asking for clarification.** "In step 4, which balloons do I actually have to check?"

Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `arrows-b.pdf`. Do **not** copy paste the chat content into your answer document.

Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Step 3. Code

(i) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(ii) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `arrows-c.pdf`. Do **not** copy paste the chat content into your answer document.

Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(d) Analysis (1 pt)

Give the big-O **runtime** and the big-O **space** of your solution, each with a one-sentence justification.

3. Assign cookies (9 pts total)

You have cookies to assign to your children. This is a huge responsibility that requires you to put your algorithms hat on.

Each child `i` has a greed factor `g[i]`, the smallest cookie size that will satisfy them. Each cookie `j` has a size `s[j]`. A child can be given at most one cookie, and cookie `j` satisfies child `i` only if `s[j] >= g[i]`.

Return the **maximum number of children** you can satisfy.

Leetcode link

Here are some example input and output:

#	`g` (children)	`s` (cookies)	answer	why
1	`[1,2,3]`	`[1,1]`	1	only the child with greed 1 can be satisfied
2	`[1,2]`	`[1,2,3]`	2	plenty of cookies, both children get fed

(a) Step 1. Brainstorm greedy strategies (1 pt)

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own.

Write down **at least two** different greedy strategies you could imagine using here. For each one, say in a single line what rule decides which cookie goes to which child.

(b) Step 2. Prove, or kill it

(i) Kill this one with a counterexample (1 pt)

A classmate proposes this strategy:

> Go through the children from **least greedy to most greedy**. Give each child the **largest** cookie that still satisfies them. If no remaining cookie satisfies them, that child goes hungry.

Find a **counterexample**: a small case where this feeds fewer children than possible. Give the greed factors and the cookie sizes, say how many children the rule feeds, and how many could have been fed.

You can do it with **two children and two cookies**.

(ii) Prove the one that works (1 pt)

The strategy that does work is:

> Go through the children from **most greedy to least greedy**. Give each child the **largest** available cookie.

To simplify the proof, let's just consider the case where we have a matching number of cookies and kids, and that there exists a matching where all the kids are satisfied. Note this is not a full proof, but it still carries the important intuition needed for the full proof.

Prove it with an **exchange argument**. Fill in this skeleton:

1. Imagine the kids being lined up left to right, in descending order of greed. So, the leftmost kid is the greediest. Let `g_1, ..., g_n` be the sizes of the cookies that get paired up with the kids according to the greedy algorithm, and let `o_1, ..., o_n` be the sizes of the cookies of a different optimal solution.
2. Walk along both until they first disagree, at position `i`. So `g_j = o_j` for every `j < i`, and `g_i != o_i`.
3. Because of how greedy chooses, we know that `g_i` ... compared to `o_i`. **Q:** What is the inequality here? >=? <=? Why? Remember that `g_i` and `o_i` refers to the size of the cookie being paired up with the `i`-th greediest kid.
4. Now, swap the assignment of cookie `o_i` with cookie `g_i`, and so we are considering OPT', which is `o_1, ..., o_{i-1}, g_i, o_{i+1}, ..., o_i, ..., o_n`. Argue that this assignment will also be valid. **Q:** For this assignment to still be valid, how do we know that the swapped cookies are good? that the `g_i` cookie will satisfy the `i`-th kid and `o_i` will satisfy the `j`-th kid, where `j` is the index of `g_i` in OPT?
5. Conclude.

Fill in steps 3 to 5.

(iii) AI plan check

Use the homework chatbot to check your counterexample from (i) and your proof from (ii). Here are some example prompts:

- **Verifying.** "Here is the problem (copy paste). Here is my counterexample and my exchange argument (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck on case 2. Can you give me a hint?"
- **Asking for clarification.** "Why does giving the smallest sufficient cookie never lose anything?"

Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `cookies-b.pdf`. Do **not** copy paste the chat content into your answer document.

Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Step 3. Code

(i) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(ii) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `cookies-c.pdf`. Do **not** copy paste the chat content into your answer document.

Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(d) Analysis (1 pt)

Give the big-O **runtime** of your solution, with a one-sentence justification.

4. Magnetic force between two balls (2 pts total)

You are given `position`, the coordinates of `n` baskets on a line, and an integer `m`. You have to put `m` balls into `m` distinct baskets. The **magnetic force** between two balls is the distance between their baskets.

Return the largest possible value of the **minimum** magnetic force between any two balls. In other words: spread the `m` balls out as evenly as you can, and report how far apart the closest pair ends up.

Leetcode link

Here are some example input and output:

#	`position`	`m`	answer	why
1	`[1,2,3,4,7]`	3	3	use baskets 1, 4 and 7; the closest pair is 3 apart
2	`[5,4,3,2,1,1000000000]`	2	999999999	use baskets 1 and 1000000000

This one is a bit different from the other three, so the questions walk you through a specific plan. **We will be lenient on the proof here** -- what matters is that you can explain the plan. There is no coding on this problem.

(a) The helper function (1 pt)

Consider a helper function `feasible(space)` that returns true if and only if you can place all `m` balls in baskets such that **every** pair of balls ends up at least `space` apart.

In 2 to 3 sentences, describe how you would implement `feasible(space)`. Be concrete about what you scan and what you count.

(b) Searching for the answer (1 pt)

Given `feasible(space)`, how do you find the **largest** `space` for which `feasible(space)` is still true?

Then: what property of `feasible()` makes that search efficient rather than a scan of every candidate? **Hint:** it may help to look back at the divide and conquer lecture.

Answer in 3 to 5 sentences. That's it! You don't have to code this one up.